

Week 2 Day 4: Docker Compose와 다중 컨테이너 실행 표준화

Overview

Day 4는 Day 3에서 배운 port, environment variable, volume, network 옵션을 `compose.yaml` 로 옮긴다. 학생은 여러 컨테이너를 손으로 하나씩 실행하는 방식에서 벗어나, 실행 조건을 파일로 관리하고 같은 project 단위로 시작, 확인, 로그 조회, 정리하는 흐름을 익힌다.

오늘의 핵심 질문은 다음과 같다.

여러 컨테이너의 실행 조건을 `compose.yaml`로 묶으면 실행, 확인, 장애 분석, 인수인계가 어떻게 달라지는가?

Compose는 Kubernetes를 대체하는 운영 플랫폼이 아니다. 로컬 개발, 실습, 작은 통합 테스트, 실행 조건 공유에 강한 도구다. 그래서 Day 4의 평가는 "Compose 파일을 작성했다"가 아니라 `docker compose config`, `up`, `ps`, `logs`, `run`, `down` evidence로 재현 가능성을 증명하는지에 둔다.

Learning Goals

- 긴 `docker run` 옵션을 Compose의 `services`, `ports`, `environment`, `volumes`, `networks` 로 mapping한다.
- `compose.yaml` 의 기본 구조와 indentation 오류를 점검한다.
- web service와 database service를 하나의 Compose project로 실행한다.
- Compose가 만드는 network와 service name 기반 DNS를 설명한다.
- `.env`, bind mount, named volume을 개발 환경 기준으로 구분한다.
- `depends_on` 과 `healthcheck` 의 의미와 한계를 설명한다.
- `docker compose config`, `ps`, `logs`, `exec`, `run` 으로 장애를 분류한다.
- README에 `compose` 실행, 확인, cleanup, failure drill을 handoff 형식으로 기록한다.

Lesson Index

- 1교시: Docker Compose가 필요한 이유 - 긴 `docker run` 옵션을 파일화
- 2교시: `compose.yaml` 기본 구조 - `services`, `ports`, `environment`, `volumes`, `networks`
- 3교시: 웹 애플리케이션 + DB Compose 실행 - `up`, `ps`, `logs`, `run`
- 4교시: Compose 네트워크와 service name - `db` DNS, `depends_on`, `healthcheck`
- 5교시: 개발 환경용 Compose 구성 - bind mount, `.env`, local-only 기준
- 6교시: Compose 장애 분석 - missing env, wrong port, stale volume
- 7교시: 개인 면담 및 환경 점검 - Dockerfile, Compose, Docker Hub readiness
- 8교시: 보충 실습 - Compose 진도 회복과 개인 프로젝트 연결

Practice Files And Assets

자료	용도
labs/compose-app/compose.yaml	web, db, db-client service 실습
labs/compose-app/.env.example	local .env 작성 기준
labs/compose-app/html/index.html	nginx bind mount HTTP 확인
labs/compose-app/db/init/001_schema.sql	PostgreSQL 초기화 evidence
labs/compose-app/README.md	실습 실행/확인/정리 요약
hands-on-lab.md	Day 4 전체 실행 흐름
assets/lesson-01-compose-why.png	docker run에서 Compose로 이동
assets/lesson-02-compose-file-map.png	Compose file 구조
assets/lesson-03-compose-up-evidence.png	up 이후 evidence

assets/lesson-04-compose-network-service-name.png	service name DNS
assets/lesson-05-dev-compose.png	개발용 bind mount와 env
assets/lesson-06-compose-troubleshooting.png	장애 분류 순서
assets/lesson-07-compose-readiness-board.png	개인 점검 보드
assets/lesson-08-compose-recovery.png	보충 실습 회복 흐름

Required Evidence

Evidence	제출 기준
Config evidence	docker compose config 성공 또는 missing env 실패를 설명
Project evidence	docker compose ps의 web/db 상태
HTTP evidence	curl -I http://localhost:18084, body marker
DB evidence	db-client 또는 exec db psql 결과
Network evidence	service name db로 접속한 결과
Volume evidence	pgdata named volume과 down -v 위험 설명
Log evidence	docker compose logs db 핵심 줄
Failure evidence	missing env, wrong port, stale volume 중 하나의 RCA
Cleanup evidence	down과 down -v 차이 기록

Official References

Topic	Reference	확인할 키워드
Docker Compose	https://docs.docker.com/compose/	multi-container, lifecycle
How Compose works	https://docs.docker.com/compose/intro/compose-application-model/	services, networks, volumes, project
Compose services	https://docs.docker.com/reference/compose-file/services/	ports, environment, env_file, depends_on, healthcheck
Compose networks	https://docs.docker.com/reference/compose-file/networks/	default network, service discovery
Networking in Compose	https://docs.docker.com/compose/how-tos/networking/	project network, service name DNS
PostgreSQL official image	https://hub.docker.com/_/postgres	env, init scripts, data directory

Readiness And Cost Notes

- Day 4는 로컬 Docker Compose 실습이다. cloud resource, 유료 DB, Kubernetes cluster를 만들지 않는다.
- .env 는 실습자의 로컬 파일로 두고, 공개 repository에는 실제 secret 값을 올리지 않는다.
- docker compose down -v 는 DB volume 삭제를 의미할 수 있다. cleanup 명령은 data 삭제 여부를 구분해 기록한다.
- port 충돌이 있으면 host port만 변경한다. container 내부 80 이나 PostgreSQL 5432 를 먼저 바꾸지 않는다.

End-Of-Day Checklist

- ☐ `docker compose config` 로 Compose file을 검증했다.
- ☐ `docker compose up -d` 로 web과 db를 실행했다.
- ☐ `docker compose ps` 에서 db health 상태를 확인했다.
- ☐ browser 또는 `curl` 로 `localhost:18084` HTTP 200을 확인했다.
- ☐ service name `db` 로 PostgreSQL에 접근했다.
- ☐ missing env 또는 wrong port failure drill을 기록했다.
- ☐ `docker compose down` 과 `down -v` 의 차이를 설명했다.
- ☐ README에 `compose run/check/cleanup handoff`를 남겼다.

Next Connection

Day 5는 Week 2 전체를 운영 관점으로 정리한다. Day 4의 Compose 경험은 Day 5에서 좋은 Dockerfile, secret 관리, 이미지 배포 흐름, 통합 실습 발표의 기준이 된다. Week 3의 MSA에서는 Compose의 service name과 network 경험이 여러 서비스 간 API 연결을 이해하는 출발점이 된다.

Prerequisites

Day 4를 시작하기 전에 학생은 다음을 확인해야 한다.

항목	확인 명령	정상 기준
Docker daemon	<code>docker version</code>	client와 server 정보가 모두 출력
Compose CLI	<code>docker compose version</code>	Compose version 출력
기본 image	<code>docker image ls</code>	nginx 또는 postgres가 없으면 pull 가능
terminal 위치	<code>pwd</code>	repository root 또는 lab directory
browser/curl	<code>curl --version</code>	HTTP 확인 가능
port 여유	18084 사용 가능	충돌 시 WEB_PORT 변경

이 표는 설치 확인이 아니라 수업 중 failure domain을 줄이기 위한 preflight다. Docker daemon이 응답하지 않으면 Compose file을 고쳐도 해결되지 않는다.

Day 3 To Day 4 Mapping

Day 4는 Day 3의 명령을 버리는 것이 아니라 같은 실행 조건을 더 재현 가능한 형식으로 옮긴다.

Day 3 runtime option	Day 4 Compose 항목	확인 evidence
<code>docker network create</code>	networks	network name, service connectivity
<code>docker volume create</code>	volumes	named volume, cleanup warning
<code>docker run -p</code>	services.web.ports	compose ps, HTTP 200
<code>docker run -e</code>	services.db.environment, .env	config, missing env drill
<code>docker run -v</code>	services.*.volumes	bind mount, named volume
<code>docker logs</code>	<code>docker compose logs SERVICE</code>	DB startup/readiness log
<code>docker exec/run</code>	<code>docker compose exec/run</code>	DB query, service DNS
<code>docker rm/network rm/volume rm</code>	<code>docker compose down, down -v</code>	cleanup audit

Academic/Workforce Standards Alignment

Day 4는 실습 중심이지만 평가 기준은 학술성과 실무성을 함께 포함한다.

기준	Day 4 적용	학생 evidence
ABET 문제 분석	긴 명령 기반 실행의 재현성 문제 분석	option mapping note
ABET 커뮤니케이션	다른 사람이 실행 가능한 README 작성	run/check/cleanup section
CS2023 Knowledge	service, network, volume, environment 개념	개념 설명 문항
CS2023 Skill	Compose CLI로 실행과 검증 수행	config/up/ps/logs output
CS2023 Disposition	secret과 data 삭제 위험을 책임 있게 다룸	.env.example, cleanup warning
NIST NICE	configuration/credential hygiene	secret 비노출 기록
Bloom Analyze	장애를 config/start/runtime/cleanup으로 분류	RCA note
SRE practice	evidence와 postmortem-style learning	failure drill과 recheck

Operational Readiness Model

Day 4의 운영 준비성은 다음 네 단계로 본다.

단계	질문	좋은 답
Start	어떻게 실행하는가	cp .env.example .env, docker compose up -d
Check	정상 상태는 무엇인가	HTTP 200, body marker, DB query
Recover	실패하면 어디를 보는가	config, ps, logs, db-client, volume
Cleanup	무엇을 지우고 무엇을 남기는가	down과 down -v 구분

실무에서는 start만 있는 문서는 부족하다. check와 cleanup이 빠지면 다음 사람이 정상 여부와 비용/데이터 위험을 판단할 수 없다.

Risk Register

위험	가능성	영향	Severity	완화
.env secret 노출	중간	높음	High	.env.example만 commit
down -v data 삭제	중간	높음	High	실습 reset 전용으로 문서화
port 충돌	높음	낮음	Medium	WEB_PORT 변경
DB readiness 오해	중간	중간	Medium	healthcheck와 query 확인
service name/localhost 혼동	높음	중간	Medium	host/container 경계 표기
Docker daemon 미실행	중간	중간	Medium	preflight에서 확인
image pull 실패	낮음	중간	Medium	사전 image 확인 또는 네트워크 점검

Evidence Quality Rubric

항목	0점	1점	2점
Config	없음	실행만 함	출력 구조 해석

Web	없음	ps만 있음	HTTP 200과 body marker
DB	없음	running만 확인	healthy/log/query 확인
Network	없음	network 이름만 기록	db service name 접근 증명
Env	없음	값만 기록	required variable과 secret 비노출 설명
Volume	없음	volume 이름만 기록	data lifecycle과 down -v 위험 설명
RCA	없음	증상만 기록	원인 후보, fix, recheck, prevention
Handoff	없음	명령만 있음	expected result와 cleanup warning 포함

Common Misconceptions

오해	바로잡기
Compose는 Dockerfile을 대체한다	Dockerfile은 image build, Compose는 runtime application model
docker compose up만 성공하면 끝이다	service별 정상 기준 확인이 필요하다
host에서 db 이름으로 접속할 수 있다	db는 Compose network 내부 service name이다
.env는 secret manager다	로컬 편의 파일이며 공개하면 위험하다
down -v는 일반 cleanup이다	named volume data를 삭제할 수 있다
depends_on은 readiness를 완전히 보장한다	healthcheck와 app retry는 별도다

Suggested Day 4 Board

수업 중 칠판 또는 공유 문서에 다음 보드를 유지한다.

```

Config:
  docker compose config

Run:
  docker compose up -d

Check:
  docker compose ps
  curl -I http://localhost:18084
  docker compose run --rm db-client

Failure:
  missing env / wrong port / stale volume

Cleanup:
  docker compose down
  docker compose down -v # data reset

```

보드는 학생이 길을 잃었을 때 돌아올 수 있는 기준점이다.

Student Submission Template

```

## Week 2 Day 4 Submission
- OS / Docker Desktop or Engine:

```

- Compose version:
- Compose file path:
- Config evidence:
- Web evidence:
- DB evidence:
- Network evidence:
- Failure drill:
- Cleanup evidence:
- README handoff update:
- Remaining blocker:

Instructor Review Checklist

- ☐ 학생이 host port와 container port를 구분한다.
- ☐ 학생이 host localhost 와 Compose service name db 를 구분한다.
- ☐ 학생이 .env.example 과 실제 .env 의 차이를 설명한다.
- ☐ 학생이 down 과 down -v 의 data lifecycle 차이를 설명한다.
- ☐ 학생이 docker compose config 를 실행 전 검증으로 사용한다.
- ☐ 학생이 web과 DB를 서로 다른 evidence로 확인한다.
- ☐ 학생이 실패를 blame이 아니라 RCA 학습으로 기록한다.

Completion Definition

Day 4 완료는 다음 조건을 모두 만족할 때 선언한다.

1. compose.yaml이 존재한다.
2. docker compose config가 통과한다.
3. web service가 HTTP 200과 body marker를 반환한다.
4. db service가 health 또는 query evidence를 가진다.
5. service name db로 접속한 evidence가 있다.
6. missing env, wrong port, stale volume 중 하나의 failure drill이 있다.
7. cleanup 명령과 data 삭제 위험이 README에 기록되어 있다.

이 정의는 Day 5 통합 실습과 Week 2 최종 평가의 기준으로 이어진다.